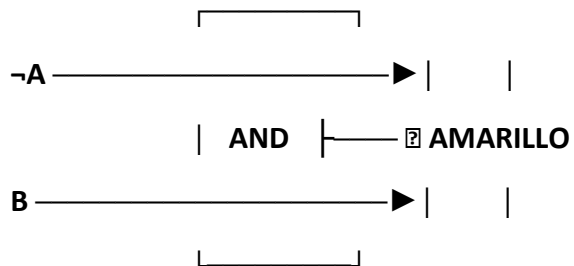
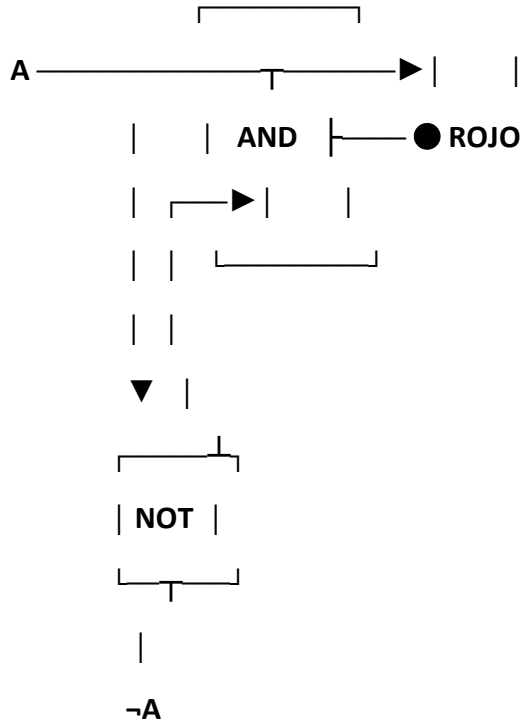


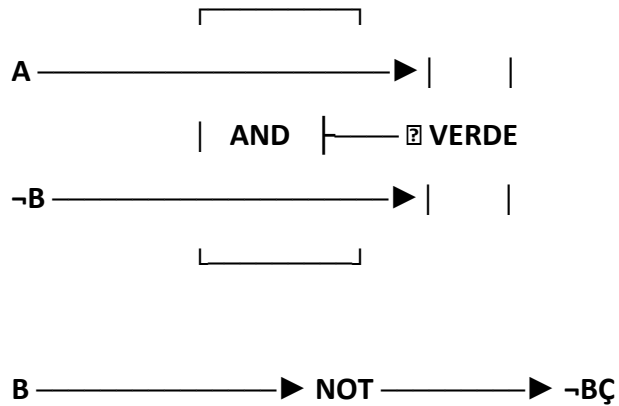
ACTIVIDAD 5

PARTE 1:

La lógica Booleana es un mundo binario de 1 y 0 donde las compuertas y tablas de verdad toman

relevancia a continuación se muestra un pequeño resumen de lo visto en clase:





2- Analice y observe la siguiente ecuación:

2. Ecuación

La ecuación de la imagen es:

$$X = [AB^-(C+BD)+A^-\overline{B}]C = [\overline{A}\overline{B}((C+BD)+\overline{A}\overline{B})]C$$

Primero podemos simplificarla:

$$X = AB^-\overline{C}X = \overline{A}\overline{B}C$$

Por lo tanto, **X será 1 únicamente cuando A=1, B=0 y C=1**. La variable D no afecta el resultado.

Tabla de verdad

A	B	C	D	X
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0

A	B	C	D	X
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Circuito lógico

Como la expresión simplificada es:

$$X = AB^{\neg}C = A\overline{B}C$$

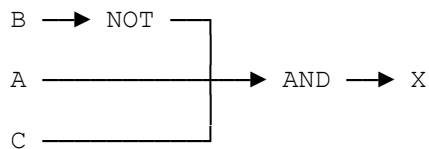
necesitas:

A → AND

B → NOT → AND

C → AND

Es decir:



3. Ecuación

La ecuación de la imagen es:

$$X = AB^{\neg}C[BD + CDE] + ACX = A\overline{B}C[BD + CDE] + AC$$

Esta también se puede simplificar.

El primer término contiene:

$$AB^{\neg}C(BD + CDE)A\overline{B}C(BD + CDE)$$

Al distribuir:

$$AB\overline{C}BD + AB\overline{C}CDEA\overline{B}CBD + A\overline{B}CCDE$$

El primer término desaparece porque:

$$B\overline{B} = 0\overline{B}B = 0$$

y el segundo queda incluido dentro de $ACAC$.

Por lo tanto:

$$X = AC \quad \boxed{X = AC}$$

Así que **X solamente depende de A y C.**

Tabla de verdad

A B C D X

0 0 0 0 0

0 0 0 1 0

0 0 1 0 0

0 0 1 1 0

0 1 0 0 0

0 1 0 1 0

0 1 1 0 0

0 1 1 1 0

1 0 0 0 0

1 0 0 1 0

1 0 1 0 1

1 0 1 1 1

1 1 0 0 0

A B C D X

1 1 0 1 0

1 1 1 0 1

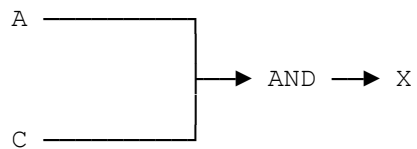
1 1 1 1 1

Circuito lógico

Como:

$X = ACX = AC$

solamente necesitamos una **compuerta AND**:



Parte 2:

En esta parte el estudiante reforzará los conceptos vistos en clase sobre Python, streamlit y

la lógica de programación aprendida de momento teniendo presente los siguientes ejercicios:

1.



↓



| |



| |

| **Seleccionar una figura** |

| |



| |

| **Ingresar las medidas** |

| |



| |

| **Calcular área y** |

| **perímetro** |

| |



| |

| **Mostrar figura,** |

| **ecuaciones y resultados** |

| |



| |

| **FIN** |

| |

```
import streamlit as st
```

```
import math
```

```
import matplotlib.pyplot as plt
```

```
from matplotlib.patches import Rectangle, Polygon, Circle
```

```
# Configuración
```

```
st.set_page_config(  
    page_title="Áreas y Perímetros",  
    page_icon="📐"  
)
```

```
st.title("📐 Cálculo de Áreas y Perímetros")
```

```
st.write("Seleccione una figura e ingrese las medidas necesarias.")
```

```
# Menú
```

```
figura = st.selectbox(  
    "Seleccione una figura:",  
    [  
        "Rectángulo",  
        "Cuadrado",  
        "Triángulo",  
        "Círculo",  
        "Paralelogramo"  
    ]  
)
```

```
# Función para mostrar las figuras
```

```
def mostrar_figura(nombre):
```

```
    fig, ax = plt.subplots(figsize=(5, 3.5))
```

```
ax.axis("off")
```

```
ax.set_aspect("equal")
```

```
# RECTÁNGULO
```

```
if nombre == "Rectángulo":
```

```
rectangulo = Rectangle(
```

```
(0, 0), 6, 3,
```

```
fill=False,
```

```
linewidth=3
```

```
)
```

```
ax.add_patch(rectangulo)
```

```
ax.text(3, -0.5, "b = base", ha="center")
```

```
ax.text(
```

```
-0.5, 1.5,
```

```
"h = altura",
```

```
rotation=90,
```

```
va="center"
```

```
)
```

```
# CUADRADO
```

```
elif nombre == "Cuadrado":
```

```
cuadrado = Rectangle(
```

```
(0, 0), 4, 4,  
fill=False,  
linewidth=3  
)
```

```
ax.add_patch(cuadrado)
```

```
ax.text(2, -0.5, "l = lado", ha="center")
```

```
# TRIÁNGULO
```

```
elif nombre == "Triángulo":
```

```
puntos = [  
(0, 0),  
(6, 0),  
(3, 4)  
]
```

```
triangulo = Polygon(  
puntos,  
closed=True,  
fill=False,  
linewidth=3  
)
```

```
ax.add_patch(triangulo)
```

```
ax.text(3, -0.5, "b = base", ha="center")
```

```
ax.text(3.2, 2, "h = altura")
```

```
# CÍRCULO
```

```
elif nombre == "Círculo":
```

```
circulo = Circle(
```

```
(0, 0),
```

```
3,
```

```
fill=False,
```

```
linewidth=3
```

```
)
```

```
ax.add_patch(circulo)
```

```
# Radio
```

```
ax.plot(
```

```
[0, 3],
```

```
[0, 0],
```

```
linewidth=2
```

```
)
```

```
ax.text(
```

```
1.5, 0.3,
```

```
"r = radio",
```

```
ha="center"
```

```
)
```

```
# PARALELOGRAMO
```

```
elif nombre == "Paralelogramo":
```

```
puntos = [
```

```
(0, 0),
```

```
(5, 0),
```

```
(7, 3),
```

```
(2, 3)
```

```
]
```

```
paralelogramo = Polygon(
```

```
puntos,
```

```
closed=True,
```

```
fill=False,
```

```
linewidth=3
```

```
)
```

```
ax.add_patch(paralelogramo)
```

```
ax.text(
```

```
2.5, -0.5,
```

```
"b = base",
```

```
ha="center"
```

```
)
```

```
ax.text(
```

```
5.5, 1.5,  
"a = lado"  
)
```

```
ax.set_xlim(-1, 8)  
ax.set_ylim(-1, 5)
```

```
st.pyplot(fig)
```

```
plt.close(fig)
```

```
# Mostrar dibujo  
mostrar_figura(figura)
```

```
# -----  
# CÁLCULOS  
# -----
```

```
if figura == "Rectángulo":
```

```
st.header("Rectángulo")
```

```
base = st.number_input(  
"Ingrese la base:",  
min_value=0.01,
```

```
value=5.0
```

```
)
```

```
altura = st.number_input(
```

```
"Ingrese la altura:",
```

```
min_value=0.01,
```

```
value=3.0
```

```
)
```

```
area = base * altura
```

```
perimetro = 2 * (base + altura)
```

```
st.subheader("Ecuaciones")
```

```
st.latex(r"A = b \times h")
```

```
st.latex(r"P = 2(b + h)")
```

```
elif figura == "Cuadrado":
```

```
st.header("Cuadrado")
```

```
lado = st.number_input(
```

```
"Ingrese el lado:",
```

```
min_value=0.01,
```

```
value=4.0
```

```
)
```

```
area = lado ** 2
```

```
perimetro = 4 * lado
```

```
st.subheader("Ecuaciones")
```

```
st.latex(r"A = l^2")
```

```
st.latex(r"P = 4l")
```

```
elif figura == "Triángulo":
```

```
st.header("Triángulo")
```

```
base = st.number_input(  
    "Ingrese la base:",  
    min_value=0.01,  
    value=6.0  
)
```

```
altura = st.number_input(  
    "Ingrese la altura:",  
    min_value=0.01,  
    value=4.0  
)
```

```
lado1 = st.number_input(  
    "Ingrese el lado 1:",  
    min_value=0.01,  
    value=3.0  
)
```

"Ingrese el lado 1:",

min_value=0.01,

value=5.0

)

lado2 = st.number_input(

"Ingrese el lado 2:",

min_value=0.01,

value=5.0

)

area = (base * altura) / 2

perimetro = base + lado1 + lado2

st.subheader("Ecuaciones")

st.latex(r"A = \frac{b \times h}{2}")

st.latex(r"P = b + l_1 + l_2")

elif figura == "Círculo":

st.header("Círculo")

radio = st.number_input(

"Ingrese el radio:",

min_value=0.01,

```
value=3.0
```

```
)
```

```
area = math.pi * radio ** 2
```

```
perimetro = 2 * math.pi * radio
```

```
st.subheader("Ecuaciones")
```

```
st.latex(r"A = \pi r^2")
```

```
st.latex(r"P = 2\pi r")
```

```
elif figura == "Paralelogramo":
```

```
st.header("Paralelogramo")
```

```
base = st.number_input(
```

```
"Ingrese la base:",
```

```
min_value=0.01,
```

```
value=5.0
```

```
)
```

```
lado = st.number_input(
```

```
"Ingrese el lado:",
```

```
min_value=0.01,
```

```
value=4.0
```

```
)
```

```
altura = st.number_input(
    "Ingrese la altura:",
    min_value=0.01,
    value=3.0
)
```

```
area = base * altura
perimetro = 2 * (base + lado)
```

```
st.subheader("Ecuaciones")
```

```
st.latex(r"A = b \times h")
```

```
st.latex(r"P = 2(b + a)")
```

```
# -----
```

```
# RESULTADOS
```

```
# -----
```

```
st.subheader("📊 Resultados")
```

```
st.success(
    f"Área = {area:.2f} unidades2"
)
```

```
st.info(
```

f"Perímetro = {perimetro:.2f} unidades"

)

1. ¿Cómo saber si un número es par o impar?

Un número es **par** cuando al dividirlo entre 2 el residuo es 0.

```
numero % 2 == 0
```

Si esta condición se cumple, el número es par. Si no se cumple, es impar.

Por ejemplo:

$8 \% 2 = 0 \rightarrow 8$ es par

$7 \% 2 = 1 \rightarrow 7$ es impar

2. ¿Cómo saber si un número es primo?

Un número **primo** es un número mayor que 1 que solamente puede dividirse exactamente entre **1 y él mismo**.

Por ejemplo:

7 es primo porque solamente es divisible entre 1 y 7.

8 no es primo porque también es divisible entre 2 y 4.

El programa prueba los posibles divisores del número para determinar si tiene alguno diferente de 1 y de él mismo.

3. ¿Cómo saber si un número es divisible por otro?

Utilizamos nuevamente %.

Si:

```
numero % divisor == 0
```

significa que la división es exacta y, por lo tanto, **el número es divisible por el divisor**.

Por ejemplo:

$$20 \% 5 = 0$$

Entonces **20 es divisible entre 5**.

Pero:

$$20 \% 3 = 2$$

Entonces **20 no es divisible entre 3**.

4. Diagrama de flujo

El diagrama que preparé sigue este proceso:

INICIO → ingresar número → comprobar par/impar → comprobar primo/no primo → ingresar divisor → comprobar divisibilidad → mostrar resultados → FIN.

 Descargar el diagrama de flujo

5. Programa en Python

 Descargar el programa actividad_2.py

El código es:

```
print("=== ANALIZADOR DE NÚMEROS ===")

numero = int(input("Ingrese un número entero: "))

# PAR O IMPAR
if numero % 2 == 0:
    print("El número es PAR.")
else:
    print("El número es IMPAR.")

# PRIMO O NO PRIMO
if numero < 2:
    primo = False
```

```

else:
    primo = True

    for i in range(2, int(numero ** 0.5) + 1):
        if numero % i == 0:
            primo = False
            break

if primo:
    print("El número es PRIMO.")
else:
    print("El número NO es PRIMO.")

# DIVISIBILIDAD
divisor = int(input(
    "Ingrese otro número para comprobar la divisibilidad: "
))

if divisor == 0:
    print("No se puede dividir entre cero.")
else:
    if numero % divisor == 0:
        print(f"{numero} ES divisible por {divisor}.")
    else:
        print(f"{numero} NO es divisible por {divisor}.")

```

Ejemplo

Si ingresamos:

Ingrese un número entero: 17

El programa responde:

El número es IMPAR.
El número es PRIMO.

Y si después ingresamos:

Ingrese otro número para comprobar la divisibilidad: 5

obtendremos:

17 NO es divisible por 5.

3. Investigación sobre YOLO

¿Qué es YOLO?

YOLO significa “**You Only Look Once**”, que en español se puede entender como “**solo miras una vez**”. Es un algoritmo de visión por computador utilizado principalmente para **detectar objetos dentro de imágenes o videos**. Por ejemplo, puede analizar una fotografía y reconocer que allí aparecen una persona, un automóvil, un perro o una bicicleta, indicando además dónde se encuentra cada objeto. El trabajo original de YOLO presentó la detección como un problema que una sola red neuronal puede resolver directamente a partir de la imagen completa.

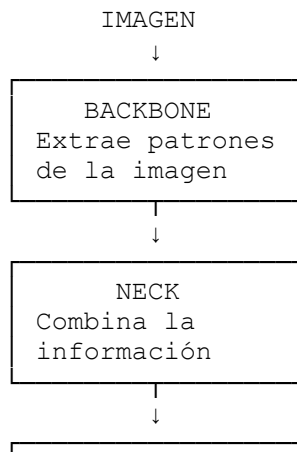
Una de las principales ventajas de YOLO es su **velocidad**, por lo que resulta especialmente útil cuando necesitamos detectar objetos prácticamente en tiempo real. Actualmente existen diferentes generaciones y variantes de YOLO. En los modelos modernos de Ultralytics, la arquitectura se organiza principalmente en tres partes: **backbone**, **neck** y **head**.

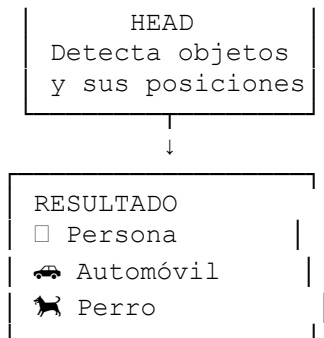
¿Cómo entiendo la arquitectura YOLO?

En palabras sencillas, entiendo la arquitectura YOLO como un sistema que recibe una imagen y la va procesando para descubrir **qué objetos aparecen y dónde están**.

El **backbone** sería como la parte del cerebro que observa la imagen y extrae sus características importantes. Por ejemplo, puede detectar formas, bordes, texturas y otros patrones que ayudan a reconocer los objetos. Después está el **neck**, que combina la información obtenida en diferentes escalas para poder reconocer tanto objetos grandes como pequeños. Finalmente está el **head**, que utiliza toda esa información para realizar las predicciones: indica la ubicación de los objetos mediante cuadros delimitadores y determina a qué clase pertenece cada uno.

De una manera todavía más sencilla se puede representar así:



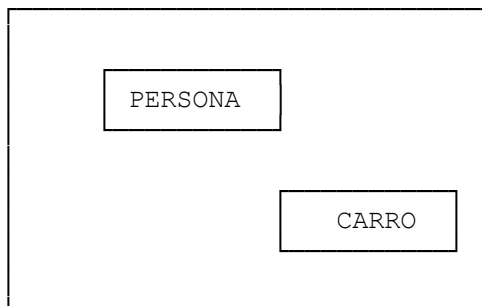


Por eso considero que YOLO es diferente de un programa tradicional en el que nosotros tenemos que indicarle exactamente dónde buscar cada objeto. En este caso, el modelo ha aprendido a reconocer diferentes características y puede realizar las detecciones automáticamente.

¿Qué hace cuando analiza una imagen?

Cuando YOLO recibe una imagen, la red neuronal procesa la información visual y produce predicciones sobre los objetos encontrados. El resultado normalmente incluye **la clase del objeto, la ubicación mediante una caja y un nivel de confianza**.

Por ejemplo, si colocamos una fotografía de una calle, podríamos obtener algo parecido a:



Esto significa que YOLO no solamente dice “**hay un carro**”, sino que también intenta indicar **en qué lugar de la imagen está el carro**.

Solución propuesta utilizando YOLO

Sistema de detección de objetos para un colegio

Una solución que plantearía sería desarrollar un **sistema de detección de objetos para mejorar la seguridad de un colegio**.

La idea sería utilizar una cámara ubicada en una entrada para analizar las imágenes y detectar determinados objetos o situaciones. Por ejemplo, el sistema podría identificar **personas, vehículos, bicicletas, mochilas u otros elementos** que previamente pueda reconocer el modelo.

El funcionamiento sería sencillo. Una cámara capturaría imágenes o video y YOLO analizaría cada cuadro. Cuando encuentre un objeto, dibujaría un cuadro alrededor de este y mostraría su nombre y nivel de confianza. Los modelos de Ultralytics están diseñados precisamente para tareas como detección de objetos y pueden utilizarse mediante Python o mediante herramientas de línea de comandos.

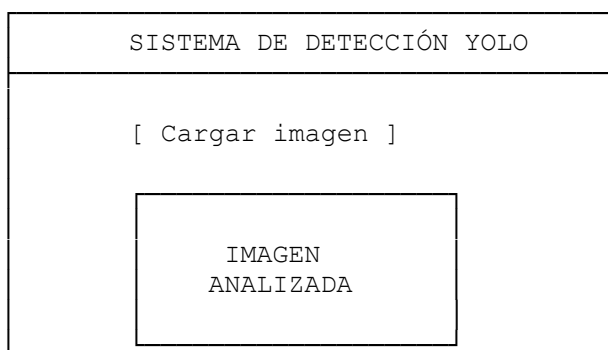
Ejemplo del funcionamiento

```
CÁMARA
↓
CAPTURA DE VIDEO
↓
YOLO ANALIZA LA IMAGEN
↓
¿ENCUENTRA OBJETOS?
↓
SÍ
↓
IDENTIFICA EL OBJETO
↓
DIBUJA EL CUADRO
↓
MUESTRA NOMBRE Y CONFIANZA
```

¿Para qué serviría?

Este proyecto podría servir para **automatizar la observación de una entrada**, generar información sobre los objetos detectados y posteriormente mostrar los resultados en una aplicación web. Por ejemplo, se podría desarrollar una interfaz con **Streamlit** donde el usuario pueda cargar una imagen y el sistema muestre automáticamente los objetos detectados.

La aplicación podría tener esta estructura:



Objetos encontrados:

✓ Persona	95%
✓ Mochila	88%
✓ Bicicleta	91%

Conclusión

En conclusión, **YOLO es una herramienta de inteligencia artificial enfocada en la visión por computador que permite detectar objetos de manera rápida en imágenes y videos**. Entiendo su arquitectura como un proceso dividido principalmente en tres partes: el **backbone** extrae las características de la imagen, el **neck** combina esa información y el **head** realiza las predicciones finales.

La solución que propongo es utilizar YOLO junto con **Python y Streamlit** para crear una aplicación capaz de cargar imágenes y detectar automáticamente los objetos que aparecen en ellas. Considero que este proyecto demuestra que la programación no solamente sirve para crear páginas web, sino que también puede utilizarse para desarrollar soluciones relacionadas con **inteligencia artificial, visión por computador y automatización**.

Si me envías ahora el enlace del repositorio que menciona la guía, te hago también la parte de “probar el algoritmo” paso a paso, con el código exacto que debes ejecutar y qué debes escribir en la evidencia de la actividad.